

Lab Work Notes

Laboratory Work — Internet Technology

Programming practice builds logic and program-developing capability, essential throughout the course. Below are worked solutions for each prescribed lab exercise, covering HTML5, CSS, JavaScript, jQuery, AJAX/JSON, and modern frameworks.

Lab 1: Simple Static Website with 4 Pages (HTML5)

Objective: Create a 4-page static site using HTML5, covering structure, text formatting, links, images, lists, tables, forms, and semantic tags.

Pages: `index.html`, `about.html`, `services.html`, `contact.html` — linked via shared nav.

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ShopEasy - Home</title>
</head>
<body>
  <header>
    <h1>ShopEasy</h1>
    <nav>
      <ul>
        <li><a href="index.html">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="services.html">Services</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section>
      <h2>Welcome to ShopEasy</h2>
      <p>Your <strong>one-stop</strong> shop for <em>quality</em> products.</p>
      
    </section>
  </main>
</body>
</html>
```

```

        </section>
    </main>

    <footer>
        <p>&copy; 2026 ShopEasy. All rights reserved.</p>
    </footer>
</body>
</html>

```

```

<!-- about.html (key section only) -->
<main>
    <article>
        <h2>About Us</h2>
        <p>We started in 2020 with one goal: quality products at fair prices.</p>
        <ol>
            <li>Founded in Kathmandu</li>
            <li>Expanded nationwide by 2023</li>
            <li>Now serving 10,000+ customers</li>
        </ol>
    </article>
</main>

```

```

<!-- services.html (key section only) -->
<main>
    <h2>Our Services</h2>
    <table border="1">
        <caption>Service Plans</caption>
        <tr><th>Plan</th><th>Price</th><th>Delivery</th></tr>
        <tr><td>Basic</td><td>Free</td><td>5-7 days</td></tr>
        <tr><td>Express</td><td>Rs. 200</td><td>1-2 days</td></tr>
    </table>
</main>

```

```

<!-- contact.html (key section only) -->
<main>
    <h2>Contact Us</h2>
    <p>Email: <a href="mailto:support@shopeasy.com">support@shopeasy.com</a></p>
    <p>Phone: <a href="tel:+9779800000000">+977-9800000000</a></p>
    <hr>
    <p>Visit us: <u>Kathmandu, Nepal</u></p>
</main>

```

Covers: doctype, head/body structure, meta tags, headings, paragraph, strong/em/u/hr, nav+ul+li, anchor tags (internal/mailto/tel), img with alt, ordered list, table with caption, semantic tags (header/nav/main/section/article/footer).

Lab 2: Simple Image Gallery Using CSS

Objective: Build responsive image gallery using CSS Grid/Flexbox, hover effects.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Image Gallery</title>
<style>
  * { box-sizing: border-box; margin: 0; padding: 0; }
  body { font-family: Arial, sans-serif; padding: 20px; }
  h1 { text-align: center; margin-bottom: 20px; }

  .gallery {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 15px;
  }
  .gallery img {
    width: 100%;
    height: 180px;
    object-fit: cover;
    border-radius: 8px;
    transition: transform 0.3s ease, box-shadow 0.3s ease;
  }
  .gallery img:hover {
    transform: scale(1.05);
    box-shadow: 0 4px 12px rgba(0,0,0,0.3);
  }
</style>
</head>
<body>
  <h1>Photo Gallery</h1>
  <div class="gallery">
    
    
    
    
    
    
  </div>
</body>
</html>
```

Covers: CSS Grid with `auto-fit / minmax` (responsive without media queries), `object-fit`, transitions, hover pseudo-class, box model (`box-sizing: border-box`).

Lab 3: Responsive Web Page Using Box Model

Objective: Demonstrate box model (content, padding, border, margin) with a responsive card layout.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Responsive Cards</title>
<style>
  * { box-sizing: border-box; }
  body { font-family: Arial, sans-serif; margin: 0; padding: 20px; background:
    #f4f4f4; }

  .card-container {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
    justify-content: center;
  }
  .card {
    width: 300px;
    padding: 20px;          /* padding - space inside border */
    border: 2px solid #2563eb; /* border */
    margin: 10px;          /* margin - space outside border */
    border-radius: 10px;
    background: white;
  }
  .card h3 { margin-bottom: 10px; }

  @media (max-width: 480px) {
    .card { width: 100%; }
  }
</style>
</head>
<body>
  <div class="card-container">
    <div class="card">
      <h3>Card One</h3>
      <p>Demonstrates padding, border, and margin working together.</p>
    </div>
    <div class="card">
      <h3>Card Two</h3>
      <p>Resizes to full width on mobile screens (see media query).</p>
    </div>
    <div class="card">
```

```

        <h3>Card Three</h3>
        <p>box-sizing: border-box keeps width consistent despite padding/border.
    </p>
</div>
</div>
</body>
</html>

```

Covers: box model layers, `box-sizing: border-box`, Flexbox wrapping, media query for responsiveness.

Lab 4: Form With All Elements, Validated via Client-Side Scripting

Objective: Build form using multiple input types, validate using JavaScript (not just HTML5 attributes) to demonstrate client-side scripting explicitly.

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Feedback Form</title>
<style>
    body { font-family: Arial, sans-serif; max-width: 500px; margin: 40px auto; }
    label { display: block; margin-top: 12px; font-weight: bold; }
    input, select, textarea { width: 100%; padding: 8px; margin-top: 4px; }
    .error { color: red; font-size: 0.85em; }
    button { margin-top: 15px; padding: 10px 20px; }
</style>
</head>
<body>
    <h2>Feedback Form</h2>
    <form id="feedbackForm" novalidate>
        <label for="name">Name:</label>
        <input type="text" id="name" name="name">
        <span class="error" id="nameError"></span>

        <label for="email">Email:</label>
        <input type="email" id="email" name="email">
        <span class="error" id="emailError"></span>

        <label for="rating">Rating (1-5):</label>
        <input type="number" id="rating" name="rating">
        <span class="error" id="ratingError"></span>

        <label for="gender">Gender:</label>
    </form>

```

```
<select id="gender" name="gender">
  <option value="">-- Select --</option>
  <option value="male">Male</option>
  <option value="female">Female</option>
</select>

<label>
  <input type="checkbox" id="subscribe" name="subscribe"> Subscribe to
  newsletter
</label>

<label for="message">Message:</label>
<textarea id="message" name="message" rows="4"></textarea>

<button type="submit">Submit</button>
</form>
```

```
<script>
```

```
document.getElementById("feedbackForm").addEventListener("submit", function(e) {
  e.preventDefault();
  let isValid = true;

  const name = document.getElementById("name").value.trim();
  const nameError = document.getElementById("nameError");
  if (name === "") {
    nameError.textContent = "Name is required.";
    isValid = false;
  } else {
    nameError.textContent = "";
  }

  const email = document.getElementById("email").value.trim();
  const emailError = document.getElementById("emailError");
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  if (!emailPattern.test(email)) {
    emailError.textContent = "Enter a valid email.";
    isValid = false;
  } else {
    emailError.textContent = "";
  }

  const rating = document.getElementById("rating").value;
  const ratingError = document.getElementById("ratingError");
  if (rating < 1 || rating > 5 || rating === "") {
    ratingError.textContent = "Rating must be between 1 and 5.";
    isValid = false;
  } else {
    ratingError.textContent = "";
  }
}
```

```

    if (isValid) {
        alert("Form submitted successfully!");
        this.reset();
    }
});
</script>
</body>
</html>

```

Covers: all form input types, `<label>`, client-side validation with JS (`addEventListener`, DOM manipulation, regex, `preventDefault`).

Lab 5: Simple Calculator Using JavaScript

Objective: Build a working calculator demonstrating variables, operators, functions, and DOM manipulation.

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Simple Calculator</title>
<style>
    body { font-family: Arial, sans-serif; display: flex; justify-content: center;
        margin-top: 50px; }
    .calculator { border: 1px solid #ccc; padding: 20px; border-radius: 8px; width:
        260px; }
    input[type="text"] { width: 100%; padding: 10px; margin-bottom: 10px; text-align:
        right; font-size: 1.2em; }
    .buttons { display: grid; grid-template-columns: repeat(4, 1fr); gap: 8px; }
    button { padding: 15px; font-size: 1em; cursor: pointer; }
</style>
</head>
<body>
    <div class="calculator">
        <input type="text" id="display" readonly>
        <div class="buttons">
            <button onclick="appendValue('7')">7</button>
            <button onclick="appendValue('8')">8</button>
            <button onclick="appendValue('9')">9</button>
            <button onclick="appendValue('/')">/</button>

            <button onclick="appendValue('4')">4</button>
            <button onclick="appendValue('5')">5</button>
            <button onclick="appendValue('6')">6</button>
            <button onclick="appendValue('*')">*</button>

```

```

        <button onclick="appendValue('1') ">1</button>
        <button onclick="appendValue('2') ">2</button>
        <button onclick="appendValue('3') ">3</button>
        <button onclick="appendValue('-') ">-</button>

        <button onclick="appendValue('0') ">0</button>
        <button onclick="clearDisplay() ">C</button>
        <button onclick="calculateResult() ">=</button>
        <button onclick="appendValue('+') ">+</button>
    </div>
</div>

<script>
const display = document.getElementById("display");

function appendValue(val) {
    display.value += val;
}

function clearDisplay() {
    display.value = "";
}

function calculateResult() {
    try {
        display.value = eval(display.value);    // simple demo only - eval() avoided in
        real production code
    } catch (error) {
        display.value = "Error";
    }
}
</script>
</body>
</html>

```

Covers: variables, functions, event handling (`onclick`), DOM manipulation, operators, `try/catch` error handling. *(Teaching note: mention `eval()` is unsafe for production — used here only for simplicity; real calculators parse expressions manually or use a math library.)*

Lab 6: User Registration Form Using HTML5 + JavaScript Validation

Objective: Combine HTML5 input types/validation attributes with JavaScript for extra logic (e.g., password match).

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>User Registration</title>
<style>
  body { font-family: Arial, sans-serif; max-width: 450px; margin: 40px auto; }
  label { display: block; margin-top: 10px; }
  input { width: 100%; padding: 8px; margin-top: 4px; }
  .error { color: red; font-size: 0.85em; }
</style>
</head>
<body>
  <h2>User Registration</h2>
  <form id="registerForm">
    <label for="username">Username:</label>
    <input type="text" id="username" name="username" required minlength="4"
      maxlength="15">

    <label for="email">Email:</label>
    <input type="email" id="email" name="email" required>

    <label for="dob">Date of Birth:</label>
    <input type="date" id="dob" name="dob" required>

    <label for="phone">Phone:</label>
    <input type="tel" id="phone" name="phone" pattern="[0-9]{10}" required>

    <label for="password">Password:</label>
    <input type="password" id="password" name="password" required minlength="8">

    <label for="confirmPassword">Confirm Password:</label>
    <input type="password" id="confirmPassword" name="confirmPassword" required>
    <span class="error" id="passwordError"></span>

    <button type="submit">Register</button>
  </form>

  <script>
document.getElementById("registerForm").addEventListener("submit", function(e) {
  const password = document.getElementById("password").value;
  const confirmPassword = document.getElementById("confirmPassword").value;
  const passwordError = document.getElementById("passwordError");

  if (password !== confirmPassword) {
    e.preventDefault(); // stops HTML5-valid form from submitting
    passwordError.textContent = "Passwords do not match!";
  } else {
    passwordError.textContent = "";
  }
}

```

```
});  
</script>  
</body>  
</html>
```

Covers: HTML5 input types (`email`, `date`, `tel`), built-in validation attributes (`required`, `minlength`, `pattern`), JS for custom logic HTML5 alone can't do (password match check).

Lab 7: jQuery Slider and Image Gallery

Objective: Use jQuery to build a simple auto-sliding image gallery.

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
<meta charset="UTF-8">  
<title>jQuery Image Slider</title>  
<style>  
  .slider { position: relative; width: 500px; height: 300px; overflow: hidden;  
    margin: 40px auto; }  
  .slider img { width: 500px; height: 300px; display: none; object-fit: cover; }  
  .slider img.active { display: block; }  
  .slider-btn {  
    position: absolute; top: 50%; transform: translateY(-50%);  
    background: rgba(0,0,0,0.5); color: white; border: none; padding: 10px; cursor:  
    pointer;  
  }  
  #prevBtn { left: 0; }  
  #nextBtn { right: 0; }  
</style>  
</head>  
<body>  
  <div class="slider">  
      
      
      
    <button class="slider-btn" id="prevBtn">&#10094;</button>  
    <button class="slider-btn" id="nextBtn">&#10095;</button>  
  </div>  
  
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>  
<script>  
$(document).ready(function() {  
  let current = 0;  
  const slides = $(".slider img");  
  
  function showSlide(index) {
```

```

        slides.removeClass("active").eq(index).addClass("active");
    }

    $("#nextBtn").click(function() {
        current = (current + 1) % slides.length;
        showSlide(current);
    });

    $("#prevBtn").click(function() {
        current = (current - 1 + slides.length) % slides.length;
        showSlide(current);
    });

    // auto-slide every 3 seconds
    setInterval(function() {
        current = (current + 1) % slides.length;
        showSlide(current);
    }, 3000);
});
</script>
</body>
</html>

```

Covers: jQuery selectors (`$()`), `.click()` , `.addClass()` / `.removeClass()` , `setInterval` , DOM class toggling for slider effect.

Lab 8: jQuery Date Picker and Sort

Objective: Use jQuery UI date picker and demonstrate sorting a list with jQuery.

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>jQuery Date Picker & Sort</title>
<link rel="stylesheet" href="https://code.jquery.com/ui/1.13.2/themes/base/jquery-
ui.css">
<style>
    body { font-family: Arial, sans-serif; margin: 40px; }
    #productList { list-style: none; padding: 0; width: 300px; }
    #productList li { padding: 10px; border: 1px solid #ddd; margin-bottom: 5px; }
</style>
</head>
<body>
    <h2>Select Delivery Date</h2>
    <label for="deliveryDate">Date:</label>
    <input type="text" id="deliveryDate">

```

```

<h2>Product List (Sort by Price)</h2>
<button id="sortAsc">Sort Ascending</button>
<button id="sortDesc">Sort Descending</button>
<ul id="productList">
  <li data-price="1500">Watch - Rs. 1500</li>
  <li data-price="500">Socks - Rs. 500</li>
  <li data-price="3000">Shoes - Rs. 3000</li>
  <li data-price="800">Cap - Rs. 800</li>
</ul>

<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script src="https://code.jquery.com/ui/1.13.2/jquery-ui.min.js"></script>
<script>
$(document).ready(function() {
  // jQuery UI datepicker - real project: booking/delivery date selection
  $("#deliveryDate").datepicker({
    dateFormat: "dd-mm-yy",
    minDate: 0    // disallow past dates
  });

  function sortList(order) {
    const items = $("#productList li").get();
    items.sort(function(a, b) {
      const priceA = parseInt($(a).data("price"));
      const priceB = parseInt($(b).data("price"));
      return order === "asc" ? priceA - priceB : priceB - priceA;
    });
    $.each(items, function(i, item) {
      $("#productList").append(item);
    });
  }

  $("#sortAsc").click(() => sortList("asc"));
  $("#sortDesc").click(() => sortList("desc"));
});
</script>
</body>
</html>

```

Covers: jQuery UI widget (`datepicker`), `.data()` , `.sort()` , `.append()` , `$.each()` — real pattern for sortable product/price lists.

Lab 9: JSON Data and AJAX Requests Using Fetch API

Objective: Fetch JSON data from a public API and render it dynamically.

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Fetch API Demo</title>
<style>
  body { font-family: Arial, sans-serif; margin: 40px; }
  .user-card { border: 1px solid #ddd; padding: 12px; margin-bottom: 10px; border-
    radius: 6px; }
  #loading { color: gray; }
</style>
</head>
<body>
  <h2>User List (from API)</h2>
  <p id="loading">Loading users...</p>
  <div id="userContainer"></div>

<script>
async function loadUsers() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!response.ok) throw new Error("Network error: " + response.status);
    const users = await response.json();    // JSON string -> JS array of objects

    document.getElementById("loading").style.display = "none";
    const container = document.getElementById("userContainer");

    container.innerHTML = users.map(user => `
      <div class="user-card">
        <h3>${user.name}</h3>
        <p>Email: ${user.email}</p>
        <p>Company: ${user.company.name}</p>
      </div>
    `).join("");
  } catch (error) {
    document.getElementById("loading").textContent = "Failed to load users: " +
      error.message;
  }
}

loadUsers();

// POST example - real project: submitting new data to a server
async function createPost(title, body) {
  const response = await fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title, body, userId: 1 })
  });
  const data = await response.json();

```

```
    console.log("Created post:", data);
  }
</script>
</body>
</html>
```

Covers: `fetch()` GET/POST, `async/await`, `try/catch`, `JSON.stringify / response.json()`, dynamic DOM rendering with `.map().join("")`.

Lab 10: General Concept of React/Angular/Vue.js

Objective: Give exposure to component-based thinking without requiring full framework setup — build a static HTML/JS mockup that mirrors a React component, then show the real React equivalent for comparison.

Step 1 — Vanilla JS version (what students already know):

```
<div id="app"></div>
<script>
function ProductCard(name, price) {
  return `
    <div class="card">
      <h3>${name}</h3>
      <p>Rs. ${price}</p>
    </div>
  `;
}

const products = [
  { name: "Shoes", price: 3000 },
  { name: "Watch", price: 5000 }
];

document.getElementById("app").innerHTML = products
  .map(p => ProductCard(p.name, p.price))
  .join("");
</script>
```

Step 2 — Same thing conceptually in React (for comparison/demo only, needs Node.js + React setup to actually run):

```
function ProductCard({ name, price }) {
  return (
    <div className="card">
      <h3>{name}</h3>
```

```

        <p>Rs. {price}</p>
      </div>
    );
  }

function App() {
  const products = [
    { name: "Shoes", price: 3000 },
    { name: "Watch", price: 5000 }
  ];
  return (
    <div>
      {products.map((p, i) => <ProductCard key={i} name={p.name} price={p.price}
      />)}
    </div>
  );
}

```

Discussion points for lab/class: - React/Vue = **component-based**: UI broken into reusable pieces (like `ProductCard` function above). - Framework automatically **re-renders** UI when data changes (no manual `innerHTML` updates needed). - Angular = full framework (routing, forms, HTTP client built-in), uses TypeScript; steeper learning curve, used in large enterprise apps. - Vue = a middle ground — simpler than Angular, more built-in structure than React. - Real industry use: React (Facebook, Instagram, Airbnb), Angular (enterprise dashboards, banking), Vue (smaller-to-medium apps, gradual adoption in existing projects).

Covers: component-based architecture concept, comparing vanilla JS DOM rendering vs framework rendering, JSX syntax overview, `key` prop concept.

Lab Summary Table

Lab	Topic	Key Skills Practiced
1	Static 4-page site (HTML5)	Structure, text formatting, links, images, lists, tables, semantic tags
2	Image gallery (CSS)	Grid, <code>object-fit</code> , transitions, hover
3	Responsive page (Box Model)	<code>box-sizing</code> , padding/border/margin, Flexbox, media query

Lab	Topic	Key Skills Practiced
4	Form + client-side validation	All input types, JS validation, regex, DOM
5	Calculator (JavaScript)	Variables, functions, events, operators
6	Registration form (HTML5 + JS)	HTML5 validation attributes + custom JS logic
7	jQuery slider/gallery	jQuery selectors, class toggling, <code>setInterval</code>
8	jQuery date picker + sort	jQuery UI widgets, <code>.sort()</code> , <code>.data()</code>
9	JSON + AJAX (fetch)	<code>fetch()</code> , <code>async/await</code> , JSON parsing, dynamic rendering
10	React/Angular/Vue overview	Component-based thinking, JSX, framework comparison